

Topic:

Fairness-Aware Constraint Satisfaction Framework for University Exam Timetabling with Dual-Role Instructor Conflict Modeling

What's New About Our Paper? (The Novelty)

Previous research had **three gaps** that this paper fills:

Gap 1: The "PhD Student Who's Also an instructor Problem"

In real universities, PhD students often *teach and monitor exams*, but they're *also enrolled as students themselves*. Old scheduling systems pretended these were two completely separate groups. This paper is the **first** to formally tell the solver

"Hey, if this person has their own exam at 10am, don't also assign them to watch over someone else's exam at 10am." We call this the **H4 constraint**.

Gap 2: Big Exams Need Multiple Rooms

If 446 students need to take the same exam, one room won't fit them. Old systems would figure out room assignments *after* the schedule was made (like an afterthought). This paper makes the computer figure out **timeslots AND rooms at the same time**, so it can make smarter decisions. We demonstrated this by successfully scheduling that 446-student exam across **14 classrooms simultaneously**.

Gap 3: Fair Workload for Instructors

If you have 40 Instructors watching exams, it's unfair if one watches 22 exams while another watches only 1. Previous systems tried to balance this as a secondary concern. This paper makes **fairness a primary goal** using something called a **min-max proxy** (explained below).

The Model we used and Why

The Tool: Google OR-Tools CP-SAT Solver

Think of this as an extremely powerful puzzle-solving engine. You give it all the rules, and it searches for a solution that satisfies them. "CP-SAT" stands for **Constraint Programming - Satisfiability**. It's particularly good at:

- Proving when a solution is *truly* the best possible (not just pretty good)
- Handling thousands of yes/no decisions simultaneously

How the Problem is Set Up (Decision Variables)

The computer makes three types of yes/no or number decisions:

Variable What it decides

X_e Which timeslot does exam e go in?

Y_{er} Does exam e use room r ? (Yes/No)

Z_{ei} Does instructor i watch exam e ? (Yes/No)

Hard Constraints (Must NEVER be broken)

- **H1:** No student can have two exams at the same time
- **H2:** A room can't have more students than seats
- **H3:** Two exams can't share the same room at the same time
- **H4 (NEW):** A PhD Student-Instructor can't be in their own exam AND watching another exam at the same time
- **H5:** A Instructor can't watch two exams at once
- **H6:** Every exam must have enough invigilators

Soft Constraints (Try to minimize, but not absolute)

These are preferences, not hard requirements. Breaking them adds a "penalty score" the goal is to keep the total penalty low:

- **S1:** Don't schedule Instructors on their days off
- **S2 (NEW):** Keep workload fair — minimize the gap between the most-loaded and least-loaded Instructor
- **S3:** Avoid giving Instructors back-to-back exam duties
- **S4:** Don't schedule a student's exams on back-to-back days
- **S5:** Don't split exams across more rooms than necessary

Why Min-Max Instead of Variance?

Previous work tried to balance workload using **variance** (a statistics concept). The paper replaces this with a simpler, stronger idea:

Min-Max Gap = (Instructor with MOST exams) – (Instructor with FEWEST exams)

Minimizing this directly targets the worst-case unfairness. Variance can be low even when one Instructor has a massively unfair load, the min-max gap cannot

hide that. Also practically, the CP-SAT solver handles min-max natively but doesn't support variance calculations.

Our Results

We tested on **4 standard benchmark datasets** (called Carter benchmarks, named after their creator) and **1 real university** (Istanbul Okan University, Fall 2024).

Benchmark Tests

Dataset Exams What Happened

hec-s-92	80	Perfect solution. Solved optimally in just 14.3 seconds. Every Instructor got exactly 9 invigilation duties gap of zero
sta-f-83	138	Good solution. Fairness gap of only 3
yor-f-83	180	Good solution. Fairness gap of only 2 , even though every single room and timeslot was completely full
ear-f-83	189	Fairness collapsed to a gap of 21 (one Instructor got 22 duties, another got 1)
uta-s-92	638	Computer ran out of memory and crashed

Why did ear-f-83 fail on fairness? The reason is because the schedule was so packed (every room and every timeslot completely full) that the computer had *no flexibility* to redistribute work. For example like trying to rearrange furniture in a room that's completely stuffed you physically can't move anything. This is called a "**structural saturation**" problem, not a flaw in our method.

Why did uta-s-92 crash? The room-clash constraint (H3) requires checking every possible pair of exams against every room. With 638 exams and 20 rooms, that's roughly about **8.1 million constraints** which is too much memory for one computer pass.

Real University Test (Istanbul Okan University)

There are 131 exams, 36 instructors (25 of whom were PhD Student-Instructors), 29 rooms, 62 timeslots.

Best result achieved:

- Zero scheduling conflicts for any student
- Zero preference violations (no Instructor scheduled on their day off)

- Fairness gap of just **2** (loads ranged from 4 to 6 duties — nearly perfect)
- Zero dual-role violations — all 25 PhD students were protected by H4
- The 446-student exam was successfully split across 14 rooms
- Giving the solver more time (600s vs 300s) improved the schedule by ~20%

The S3 Experiment (Ablation Study)

We also tested what happens when you also try to prevent back-to-back Instructor duties (S3). The result was striking enabling S3 made *everything worse*:

Metric	Without S3	With S3
Instructor preference violations	0	20
Fairness gap	2	9
Overall score	2,630	5,898 (+124%)

Why? S3 requires creating about **236,000 extra yes/no variables** in the solver's mind. That overwhelms it within the time limit, so it can't optimize anything well. The lesson: sometimes adding more goals makes everything worse if the computer can't handle the complexity.

Short summary of our paper

Our paper built a smarter exam scheduling system that, for the first time, handles the "PhD student who's also an instructor" conflict as a real rule (not an afterthought), assigns rooms and timeslots together intelligently, and treats fairness as a primary goal rather than a nice-to-have. It works very well for universities with up to ~300 exams, and was validated on real data from Istanbul Okan University